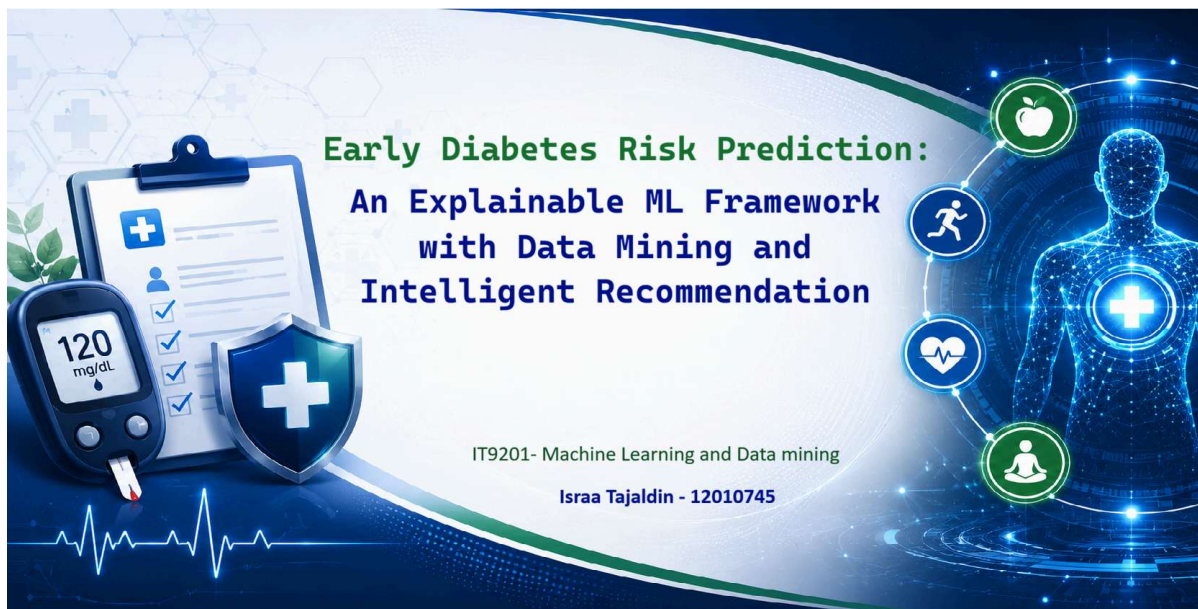




Dataset: BRFSS 2015 Diabetes Binary (Kaggle)

Models: Logistic Regression · Random Forest · XGBoost

Novelty: SHAP-based explainability + Feature-Group Hypothesis Testing + Risk-Score Recommendation Engine

```
# — Import all libraries —
# centralising imports makes dependency tracking easier and avoids duplication

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

# Scikit-learn
from sklearn.model_selection import train_test_split, StratifiedKFold, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, roc_curve, confusion_matrix, classification_report,
    ConfusionMatrixDisplay
)
from sklearn.feature_selection import mutual_info_classif

# XGBoost
from xgboost import XGBClassifier

# SHAP for explainability
import shap

# Reproducibility seed
SEED = 42
np.random.seed(SEED)

# Plot style
sns.set_theme(style='whitegrid', palette='Set2', font_scale=1.1)
plt.rcParams['figure.dpi'] = 120

print('Environment setup complete.')
print(f'pandas {pd.__version__} | numpy {np.__version__}')

Environment setup complete.
pandas 2.2.2 | numpy 2.0.2
```

TASK 1 — Problem Definition & Environment Setup

This project addresses how to improve diabetes risk prediction by integrating clinical, lifestyle, and socioeconomic features within a single machine learning framework, investigating whether this combined approach outperforms clinical features alone.

Objectives

1. Develop and compare three supervised classifiers models for diabetes binary classification using python.
2. Apply feature selection and feature group analysis to validate the project hypothesis.
3. Use Orange to perform unsupervised clustering and map discovered profiles to diabetes risk levels.
4. Optimize model hyperparameters and interpret the effect of tuning on performance.
5. Apply SHAP explainability and a risk recommendation layer to produce actionable clinical outputs.

Task 2 - Data Acquisition, Description & Pre-Processing

Data Acquisition

The dataset used in this project was obtained from Kaggle and is based on the Behavioral Risk Factor Surveillance System (BRFSS) 2015 health indicators survey maintained by the Centers for Disease Control and Prevention (CDC). The selected dataset is the **BRFSS 2015 Diabetes Binary 50/50 Balanced Dataset**, which was chosen because it provides a balanced target variable and includes a diverse set of clinical, lifestyle, and socioeconomic factors related to diabetes risk.

To facilitate development and experimentation within Google Colab, the dataset was stored in Google Drive and loaded directly into the notebook environment. This approach enabled convenient access to the data, simplified file management, and ensured reproducibility throughout the project.

```
# — Load dataset from Google Drive —————
# Reason: Mounting Google Drive is the most reliable way to load large datasets
# in Google Colab – it avoids URL timeouts, rate limits, and re-downloading.

from google.colab import drive
import os, pandas as pd

# Mount Google Drive (opens an auth popup the first time)
drive.mount('/content/drive', force_remount=False)

# — Path configuration —————
DATA_PATH = '/content/drive/MyDrive/Colab Notebooks/IT9201_Project/diabetes_binary_5050split_health_indicators_BRFSS2015.csv'

# — Load dataset —————
df = pd.read_csv(DATA_PATH)

print(f'Drive mounted : /content/drive')
print(f'File loaded from: {DATA_PATH}')
print(f'Dataset shape : {df.shape}')
print(f'Columns : {df.columns.tolist()}')
df.head(5)
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)
Drive mounted : /content/drive
File loaded from: /content/drive/MyDrive/Colab Notebooks/IT9201_Project/diabetes_binary_5050split_health_indicators_BRFSS2015.csv
Dataset shape : (70692, 22)
Columns : ['Diabetes_binary', 'HighBP', 'HighChol', 'CholCheck', 'BMI', 'Smoker', 'Stroke', 'HeartDiseaseorAttack', 'PhysActivity', 'Fruits', 'Vegetables', 'Walking', 'Nuts', 'TotalFruits', 'TotalVegetables', 'TotalWalking', 'TotalNuts', 'TotalFruitsVegetablesWalkingNuts', 'TotalFruitsVegetablesWalkingNuts', 'TotalFruitsVegetablesWalkingNuts', 'TotalFruitsVegetablesWalkingNuts', 'TotalFruitsVegetablesWalkingNuts', 'TotalFruitsVegetablesWalkingNuts']
```

	Diabetes_binary	HighBP	HighChol	CholCheck	BMI	Smoker	Stroke	HeartDiseaseorAttack	PhysActivity	Fruits	Vegetables	Walking	Nuts	TotalFruits	TotalVegetables	TotalWalking	TotalNuts	TotalFruitsVegetablesWalkingNuts	TotalFruitsVegetablesWalkingNuts	TotalFruitsVegetablesWalkingNuts	TotalFruitsVegetablesWalkingNuts	TotalFruitsVegetablesWalkingNuts	TotalFruitsVegetablesWalkingNuts
0	0.0	1.0	0.0	1.0	26.0	0.0	0.0	0.0	1.0	0.0	...	...	...	...	...	...	...	...	...	...	...	...	
1	0.0	1.0	1.0	1.0	26.0	1.0	1.0	0.0	0.0	1.0	...	...	...	...	...	...	...	...	...	...	...	...	
2	0.0	0.0	0.0	1.0	26.0	0.0	0.0	0.0	1.0	1.0	...	...	...	...	...	...	...	...	...	...	...	...	
3	0.0	1.0	1.0	1.0	28.0	1.0	0.0	0.0	1.0	1.0	...	...	...	...	...	...	...	...	...	...	...	...	
4	0.0	0.0	0.0	1.0	29.0	1.0	0.0	0.0	1.0	1.0	...	...	...	...	...	...	...	...	...	...	...	...	

5 rows × 22 columns

Dataset Description

The dataset contains **70,692 survey records**, **21 predictor variables**, and one binary target variable (**Diabetes_binary**), where **0 = No Diabetes** and **1 = Prediabetes or Diabetes**. The features include clinical indicators such as BMI, HighBP, and HighChol, lifestyle variables such as physical activity and smoking status, and socioeconomic factors including education and income levels.

The diversity of feature types makes the dataset particularly suitable for investigating the project hypothesis that diabetes risk is influenced by a combination of clinical, lifestyle, and socioeconomic characteristics rather than medical factors alone.

```
# — Dataset overview —
# Reason: Systematic profiling before any transformation prevents data leakage
# and reveals issues (missing values, type mismatches, outliers)
```

```
print('=== BASIC INFO ===')
print(df.info())
print('\n=== MISSING VALUES ===')
print(df.isnull().sum().sum(), 'total missing values')
print('\n=== CLASS BALANCE ===')
vc = df['Diabetes_binary'].value_counts()
print(vc)
print(f'Class ratio: {vc[0]/vc[1]:.3f}')
```

```
=== BASIC INFO ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 70692 entries, 0 to 70691
Data columns (total 22 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Diabetes_binary        70692 non-null float64
1   HighBP                 70692 non-null float64
2   HighChol               70692 non-null float64
3   CholCheck              70692 non-null float64
4   BMI                    70692 non-null float64
5   Smoker                 70692 non-null float64
6   Stroke                 70692 non-null float64
7   HeartDiseaseorAttack  70692 non-null float64
8   PhysActivity           70692 non-null float64
9   Fruits                 70692 non-null float64
10  Veggies                70692 non-null float64
11  HvyAlcoholConsump      70692 non-null float64
12  AnyHealthcare          70692 non-null float64
13  NoDocbcCost            70692 non-null float64
14  GenHlth                70692 non-null float64
15  MentHlth               70692 non-null float64
16  PhysHlth               70692 non-null float64
17  DiffWalk               70692 non-null float64
18  Sex                    70692 non-null float64
19  Age                    70692 non-null float64
20  Education              70692 non-null float64
21  Income                 70692 non-null float64
dtypes: float64(22)
memory usage: 11.9 MB
None

=== MISSING VALUES ===
0 total missing values

=== CLASS BALANCE ===
Diabetes_binary
0.0    35346
1.0    35346
Name: count, dtype: int64
Class ratio: 1.000
```

```
# — Statistical summary —
# Reason: Detects BMI outliers and ordinal feature ranges before modelling
```

```
stats = df.describe().T
stats['skewness'] = df.skew()
stats[['mean', 'std', 'min', 'max', 'skewness']].round(2)
```

	mean	std	min	max	skewness
Diabetes_binary	0.50	0.50	0.0	1.0	0.00
HighBP	0.56	0.50	0.0	1.0	-0.26
HighChol	0.53	0.50	0.0	1.0	-0.10
CholCheck	0.98	0.16	0.0	1.0	-6.12
BMI	29.86	7.11	12.0	98.0	1.72
Smoker	0.48	0.50	0.0	1.0	0.10
Stroke	0.06	0.24	0.0	1.0	3.63
HeartDiseaseorAttack	0.15	0.35	0.0	1.0	1.98
PhysActivity	0.70	0.46	0.0	1.0	-0.89
Fruits	0.61	0.49	0.0	1.0	-0.46
Veggies	0.79	0.41	0.0	1.0	-1.41
HvyAlcoholConsump	0.04	0.20	0.0	1.0	4.52
AnyHealthcare	0.95	0.21	0.0	1.0	-4.39
NoDocbcCost	0.09	0.29	0.0	1.0	2.78
GenHlth	2.84	1.11	1.0	5.0	0.17
MentHlth	3.75	8.16	0.0	30.0	2.39
PhysHlth	5.81	10.06	0.0	30.0	1.66
DiffWalk	0.25	0.43	0.0	1.0	1.14
Sex	0.46	0.50	0.0	1.0	0.17
Age	8.58	2.85	1.0	13.0	-0.55
Education	4.92	1.03	1.0	6.0	-0.68
Income	5.70	2.18	1.0	8.0	-0.65

▼ Data Exploration and Pre-Processing

Before model development, an initial exploratory analysis was performed to understand the structure and quality of the dataset. This included examining the dataset shape, feature data types, summary statistics, missing values, class distribution, and feature distributions. These steps helped verify data integrity and identify any preprocessing requirements.

The preprocessing stage focused on preparing the data for machine learning analysis. Feature groups were organized according to their clinical, lifestyle, and socioeconomic categories to support hypothesis testing. The dataset was then split into training and testing subsets, and scaling was applied where required by specific algorithms. These preparation steps ensured that the data was suitable for feature selection, classification modelling, clustering, and performance evaluation.

▼ EDA and data mining exploration

The following plots are included to understand the data before modelling. This is important because model scores without data understanding can be misleading.

```
# — Target distribution —
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

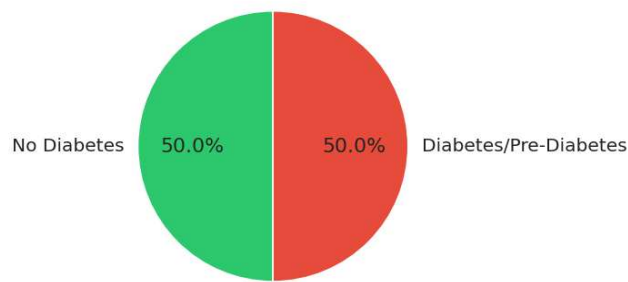
colors = ['#2ecc71', '#e74c3c']
labels = ['No Diabetes', 'Diabetes/Pre-Diabetes']

axes[0].pie(vc, labels=labels, autopct='%1.1f%%', colors=colors, startangle=90)
axes[0].set_title('Class Distribution (Balanced Binary Dataset)')

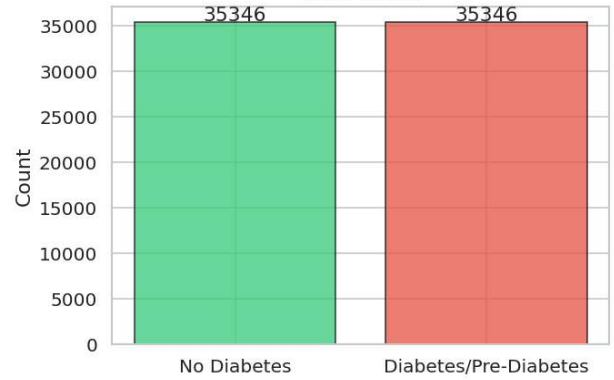
axes[1].bar(labels, vc.values, color=colors, edgecolor='black', alpha=0.7)
axes[1].set_title('Class Count')
axes[1].set_ylabel('Count')
for i, v in enumerate(vc.values):
    axes[1].text(i, v + 200, str(v), ha='center')

plt.tight_layout()
plt.savefig('fig_class_balance.png', bbox_inches='tight')
plt.show()
print('Class balance – confirms 50/50 split, no resampling needed')
```

Class Distribution (Balanced Binary Dataset)



Class Count



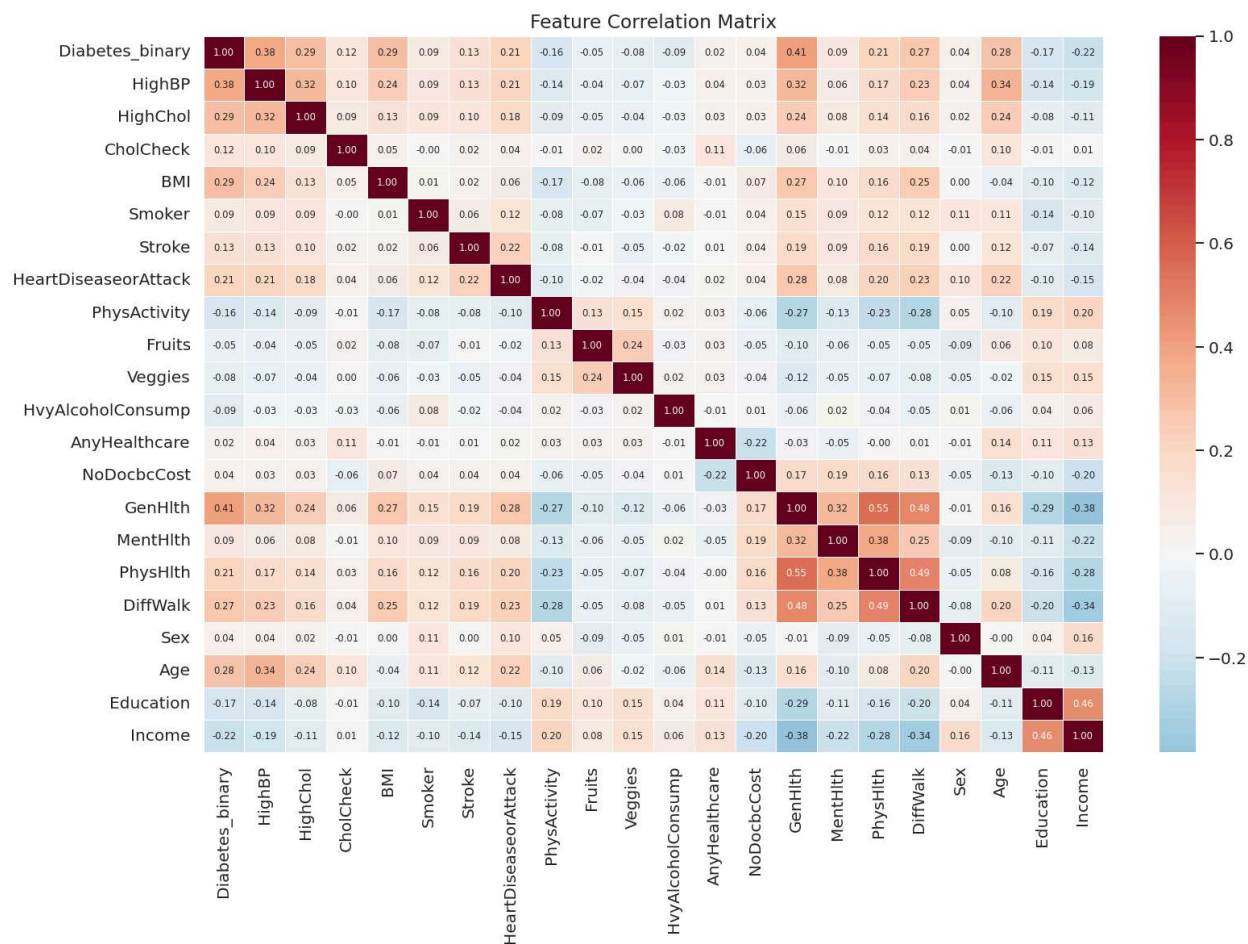
Class balance – confirms 50/50 split, no resampling needed

```
# — Correlation heatmap —
# Reason: Identifies multicollinearity; high correlation between features
# can inflate variance in linear models (Logistic Regression in particular)

fig, ax = plt.subplots(figsize=(14, 10))
corr_matrix = df.corr()

sns.heatmap(
    corr_matrix, annot=True, fmt='.2f',
    cmap='RdBu_r', center=0, linewidths=0.4,
    annot_kws={'size': 7}, ax=ax
)

ax.set_title('Feature Correlation Matrix', fontsize=14)
plt.tight_layout()
plt.savefig('fig_correlation.png', bbox_inches='tight')
plt.show()
```



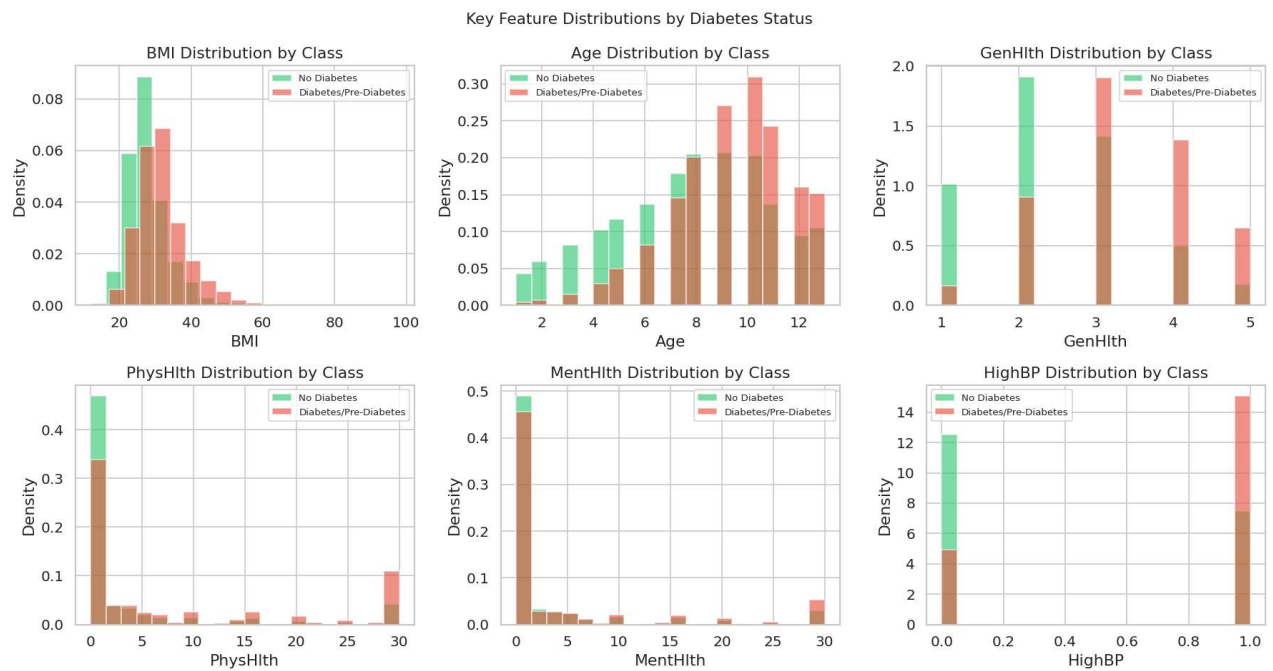
```
# — Feature distributions by class —
# Reason: Visual inspection identifies the most discriminating features before running formal selection

key_features = ['BMI', 'Age', 'GenHlth', 'PhysHlth', 'MentHlth', 'HighBP']

fig, axes = plt.subplots(2, 3, figsize=(15, 8))
axes = axes.flatten()

for i, feat in enumerate(key_features):
    for cls, color, label in zip([0, 1], colors, labels):
        subset = df[df['Diabetes_binary'] == cls][feat]
        axes[i].hist(subset, bins=20, alpha=0.6, color=color, label=label, density=True)
    axes[i].set_title(f'{feat} Distribution by Class')
    axes[i].set_xlabel(feat)
    axes[i].set_ylabel('Density')
    axes[i].legend(fontsize=8)

plt.suptitle('Key Feature Distributions by Diabetes Status', fontsize=13)
plt.tight_layout()
plt.savefig('fig_distributions.png', bbox_inches='tight')
plt.show()
```

```
# — Data Pre-Processing —
# Reason for each step:
# 1. No missing values → skip imputation (confirmed above)
# 2. BMI capping at 99th percentile → removes extreme outliers (BMI>60 are data entry errors)
# 3. StandardScaler on numeric features → required for Logistic Regression convergence;
#    tree-based models are scale-invariant but we apply globally for consistency

# Step 1: Cap BMI outliers
bmi_cap = df['BMI'].quantile(0.99)
df['BMI'] = df['BMI'].clip(upper=bmi_cap)
print(f'BMI capped at {bmi_cap:.1f} (99th percentile)')

# Step 2: Separate features and target
X = df.drop('Diabetes_binary', axis=1)
y = df['Diabetes_binary'].astype(int)

FEATURE_NAMES = X.columns.tolist()

# Step 3: Define feature groups (for hypothesis testing)
CLINICAL_FEATURES = ['HighBP', 'HighChol', 'CholCheck', 'BMI', 'Stroke', 'HeartDiseaseorAttack', 'GenHlth', 'PhysHlth', 'DiffWalk']
LIFESTYLE_FEATURES = ['Smoker', 'PhysActivity', 'Fruits', 'Veggies', 'HvyAlcoholConsump', 'MentHlth']
SOCIOECO_FEATURES = ['Income', 'Education', 'AnyHealthcare', 'NoDocbcCost', 'Sex', 'Age']
ALL_FEATURES = CLINICAL_FEATURES + LIFESTYLE_FEATURES + SOCIOECO_FEATURES

feature_sets = {
    'Clinical only': CLINICAL_FEATURES,
    'Lifestyle only': LIFESTYLE_FEATURES,
    'Socioeconomic only': SOCIOECO_FEATURES,
    'Combined': ALL_FEATURES
}

for name, cols in feature_sets.items():
    print(name, len(cols), cols)
```

```
BMI capped at 52.0 (99th percentile)
Clinical only 9 ['HighBP', 'HighChol', 'CholCheck', 'BMI', 'Stroke', 'HeartDiseaseorAttack', 'GenHlth', 'PhysHlth', 'DiffWalk']
Lifestyle only 6 ['Smoker', 'PhysActivity', 'Fruits', 'Veggies', 'HvyAlcoholConsump', 'MentHlth']
Socioeconomic only 6 ['Income', 'Education', 'AnyHealthcare', 'NoDocbcCost', 'Sex', 'Age']
Combined 21 ['HighBP', 'HighChol', 'CholCheck', 'BMI', 'Stroke', 'HeartDiseaseorAttack', 'GenHlth', 'PhysHlth', 'DiffWalk',
```

```
# ----- Train-test split (stratified to preserve class balance)-----
# Reason: Stratification ensures the 50/50 class ratio is maintained in both splits
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=SEED, stratify=y
)

# Scale - StandardScaler is fitted on the training set and applied to both sets
# Apply Standard Scaling to place numerical features on a comparable scale.
# This is important for scale-sensitive methods such as Logistic Regression
# and K-Means clustering, where larger numerical ranges can dominate learning

scaler = StandardScaler()
X_train_sc = pd.DataFrame(scaler.fit_transform(X_train), columns=FEATURE_NAMES)
X_test_sc = pd.DataFrame(scaler.transform(X_test), columns=FEATURE_NAMES)

print(f'Train: {X_train.shape} | Test: {X_test.shape}')
print(f'Class balance in train: {y_train.value_counts().to_dict()}')
```

```
Train: (56553, 21) | Test: (14139, 21)
Class balance in train: {1: 28277, 0: 28276}
```

Task 3 - Feature Selection, ML Models & Evaluation

Feature Selection Strategy

We use **Mutual Information (MI)** rather than correlation because:

- MI captures non-linear relationships (appropriate for tree-based models)
- Many features are binary/ordinal — Pearson correlation assumes linearity
- MI has been validated for health-data feature selection (Brown et al., 2012)

```
# — Mutual Information Feature Selection —
# Reason: MI scores quantify how much knowing a feature reduces uncertainty about the target.
# This is model-agnostic – useful as a pre-filter before any algorithm-specific selection.

mi_scores = mutual_info_classif(X_train, y_train, random_state=SEED)
mi_series = pd.Series(mi_scores, index=FEATURE_NAMES).sort_values(ascending=False)
```

```
# — Feature Selection: Mutual Information —

# Create feature to domain mapping from the defined feature groups
feature_domain = {}

for feature in CLINICAL_FEATURES:
    feature_domain[feature] = 'Clinical'

for feature in LIFESTYLE_FEATURES:
    feature_domain[feature] = 'Lifestyle'

for feature in SOCIOECO_FEATURES:
    feature_domain[feature] = 'Socioeconomic'

# Create ranked Mutual Information table with domain
mi_table = (
    mi_series
    .reset_index()
    .rename(columns={'index': 'Feature', 0: 'MI Score'})
)

mi_table['Domain'] = mi_table['Feature'].map(feature_domain)
mi_table['MI Score'] = mi_table['MI Score'].round(4)

mi_table = mi_table[['Feature', 'MI Score', 'Domain']]

display(mi_table)
```


	Feature	MI Score	Domain	
0	GenHlth	0.0939	Clinical	
1	HighBP	0.0836	Clinical	
2	BMI	0.0586	Clinical	
3	Age	0.0458	Socioeconomic	
4	HighChol	0.0451	Clinical	
5	DiffWalk	0.0404	Clinical	
6	PhysHlth	0.0277	Clinical	
7	HeartDiseaseorAttack	0.0256	Clinical	
8	Income	0.0252	Socioeconomic	
9	PhysActivity	0.0195	Lifestyle	
10	Education	0.0175	Socioeconomic	
11	CholCheck	0.0140	Clinical	
12	Veggies	0.0086	Lifestyle	
13	Smoker	0.0072	Lifestyle	
14	AnyHealthcare	0.0069	Socioeconomic	
15	Fruits	0.0068	Lifestyle	
16	Stroke	0.0056	Clinical	
17	Sex	0.0046	Socioeconomic	
18	HvyAlcoholConsump	0.0022	Lifestyle	
19	MentHlth	0.0006	Lifestyle	
20	NoDocbcCost	0.0001	Socioeconomic	

Next steps:

[Generate code with mi_table](#)[New interactive sheet](#)

▼ Mutual Information Threshold Selection

The **top 10** features were selected based on their Mutual Information scores. This method was used to retain the most informative predictors while keeping the feature set focused and interpretable. Selecting a fixed number of features also supports a fair comparison between models during the machine learning stage.

```
# — Mutual Information Feature Selection: Highlight Top 10 —
# Reason: Display all features while clearly highlighting the ten most informative predictors.

TOP_N = 10
top_10_features = mi_series.head(TOP_N).index.tolist()

fig, ax = plt.subplots(figsize=(10, 7))

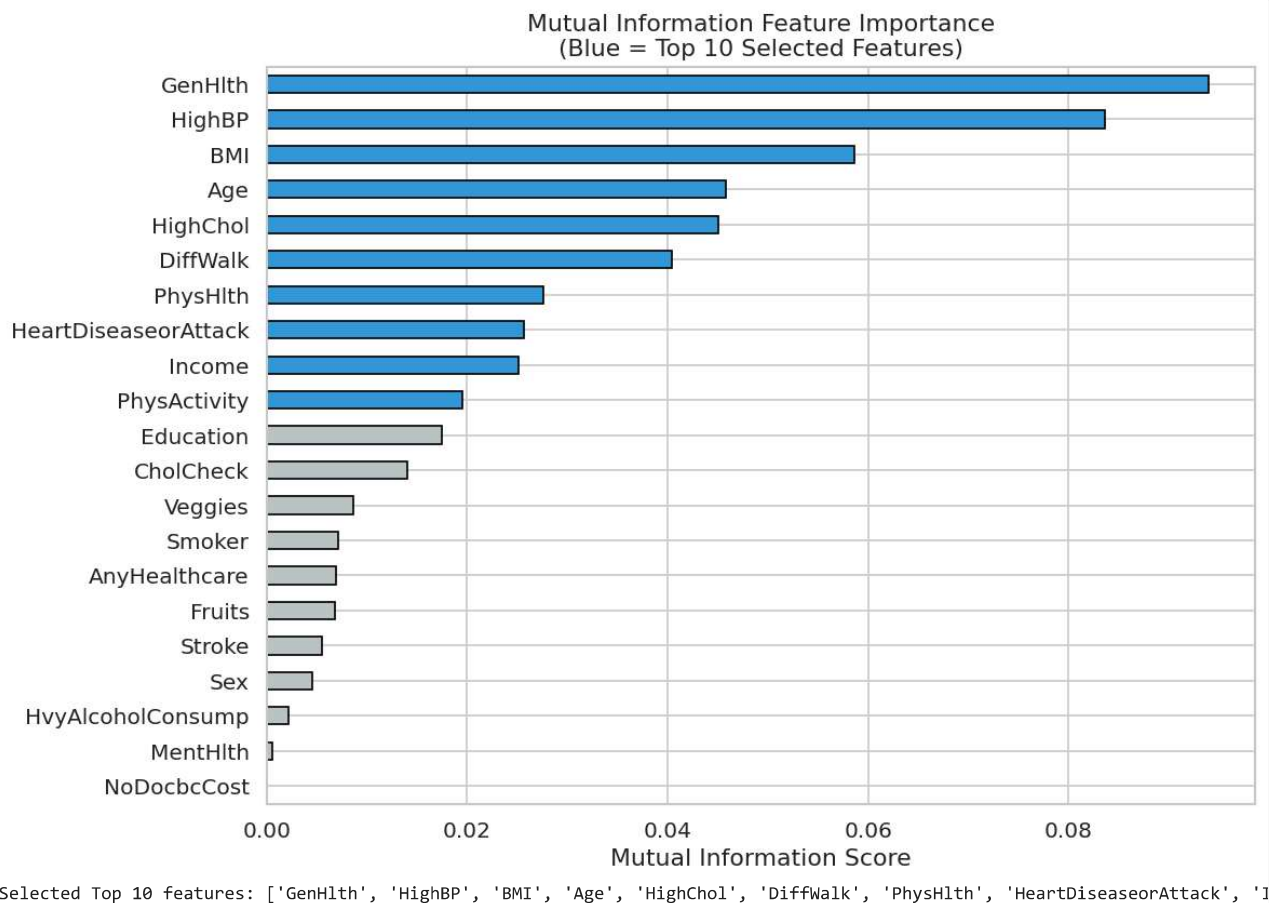
# Color Top 10 features in blue and the remaining features in grey
colors_mi = [
    '#3498db' if feature in top_10_features else '#bdc3c7'
    for feature in mi_series.index
]

mi_series.sort_values().plot(
    kind='barh',
    ax=ax,
    color=colors_mi[::-1],
    edgecolor='black'
)

ax.set_xlabel('Mutual Information Score')
ax.set_title('Mutual Information Feature Importance\n(Blue = Top 10 Selected Features)')
plt.tight_layout()
plt.savefig('fig_mutual_info_top10_highlighted.png', bbox_inches='tight')
plt.show()

# Select Top 10 features
SELECTED_FEATURES = top_10_features

print(f'Selected Top {TOP_N} features: {SELECTED_FEATURES}')
```



```
selected_mi_table = mi_table.head(10)
```

```
display(selected_mi_table)
```

	Feature	MI Score	Domain	
0	GenHlth	0.0939	Clinical	
1	HighBP	0.0836	Clinical	
2	BMI	0.0586	Clinical	
3	Age	0.0458	Socioeconomic	
4	HighChol	0.0451	Clinical	
5	DiffWalk	0.0404	Clinical	
6	PhysHlth	0.0277	Clinical	
7	HeartDiseaseorAttack	0.0256	Clinical	
8	Income	0.0252	Socioeconomic	
9	PhysActivity	0.0195	Lifestyle	

Next steps: [Generate code with selected_mi_table](#) [New interactive sheet](#)

Machine Learning Training Setup

The ML stage compares two feature sets: **clinical only** features as the baseline, and **the top 10 selected combined** features to test whether lifestyle and socioeconomic factors improve prediction.

Logistic Regression, Random Forest, and XGBoost were trained using the same split and metrics for fair comparison.

```
# — ML Experiment Setup —
# Reason: Compare clinical-only prediction against selected combined-domain features
# to test the project hypothesis.

CLINICAL_ONLY_FEATURES = CLINICAL_FEATURES
COMBINED_SELECTED_FEATURES = SELECTED_FEATURES

experiment_feature_sets = {
    'Clinical Only': CLINICAL_ONLY_FEATURES,
    'Top 10 Combined Selected': COMBINED_SELECTED_FEATURES
}
```

```
experiment_summary = pd.DataFrame({
    "Experiment": experiment_feature_sets.keys(),
    "Number of Features": [len(v) for v in experiment_feature_sets.values()],
    "Features": ["", ".join(v) for v in experiment_feature_sets.values()]
})

display(experiment_summary)
```

	Experiment	Number of Features	Features	
0	Clinical Only	9	HighBP, HighChol, CholCheck, BMI, Stroke, Hear...	
1	Top 10 Combined Selected	10	GenHlth, HighBP, BMI, Age, HighChol, DiffWalk,...	

Next steps: [Generate code with experiment_summary](#) [New interactive sheet](#)

```
# — Define Machine Learning Models —
# Reason: Three classification models are selected to compare different learning approaches:
# 1- Logistic Regression as an interpretable baseline
# 2- Random Forest as an ensemble model,
# 3- XGBoost as a high-performance boosting model for tabular data.

models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=SEED),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=SEED, n_jobs=-1),
    'XGBoost': XGBClassifier(
        n_estimators=100,
        learning_rate=0.1,
        max_depth=3,
        eval_metric='logloss',
        random_state=SEED,
        n_jobs=-1
    )
}
```

✓ Classifier Performance Evaluation

The trained classifiers were evaluated using Accuracy, Precision, Recall, F1 Score, and ROC AUC. These metrics were selected to provide a balanced view of model performance. Accuracy shows overall correctness, Precision measures how reliable positive diabetes predictions are, Recall measures how well the model identifies actual diabetes cases, F1 Score balances Precision and Recall, and ROC AUC evaluates the model's ability to separate the two classes across different thresholds.

For this healthcare problem, Recall and ROC AUC are especially important because missing high risk individuals may delay early intervention.

```
# — Train and Evaluate Models —
# Reason: Train each model on both feature sets to compare the clinical baseline
# against the selected combined feature set.

ml_results = []
trained_models = {}
predictions = {}

for feature_set_name, features in experiment_feature_sets.items():
    for model_name, model in models.items():
        model.fit(X_train[features], y_train)

        preds = model.predict(X_test[features])
        proba = model.predict_proba(X_test[features])[:, 1]

        key = f"{feature_set_name} - {model_name}"

        trained_models[key] = model
        predictions[key] = {
```

```

        "features": features,
        "y_pred": preds,
        "y_proba": proba
    }

    ml_results.append({
        'Feature Set': feature_set_name,
        'Model': model_name,
        'Accuracy': accuracy_score(y_test, preds),
        'Precision': precision_score(y_test, preds),
        'Recall': recall_score(y_test, preds),
        'F1-Score': f1_score(y_test, preds),
        'ROC-AUC': roc_auc_score(y_test, proba)
    })

ml_results_df = pd.DataFrame(ml_results).round(4)

display(
    ml_results_df.style.highlight_max(
        subset=['Accuracy', 'Precision', 'Recall', 'F1-Score', 'ROC-AUC'],
        color='lightgreen',
        axis=0
    )
)

```

	Feature Set	Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
0	Clinical Only	Logistic Regression	0.733300	0.728000	0.744900	0.736300	0.809100
1	Clinical Only	Random Forest	0.712900	0.706700	0.727700	0.717000	0.776500
2	Clinical Only	XGBoost	0.737800	0.721000	0.775900	0.747400	0.813400
3	Top 10 Combined Selected	Logistic Regression	0.742500	0.734500	0.759400	0.746700	0.819400
4	Top 10 Combined Selected	Random Forest	0.711600	0.703100	0.732500	0.717500	0.779000
5	Top 10 Combined Selected	XGBoost	0.746100	0.726300	0.789800	0.756700	0.825500

```

# — Confusion Matrix Evaluation —
# Reason: Display all confusion matrices together for easier comparison
# across models and feature sets.

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

n_models = len(predictions)

fig, axes = plt.subplots(2, 3, figsize=(15, 8))
axes = axes.flatten()

for ax, (key, values) in zip(axes, predictions.items()):
    cm = confusion_matrix(y_test, values["y_pred"])

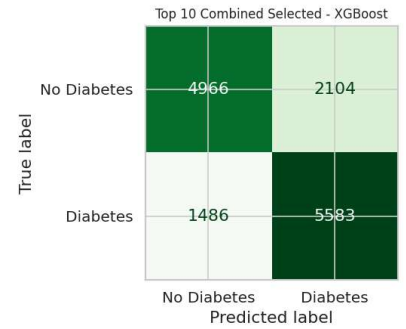
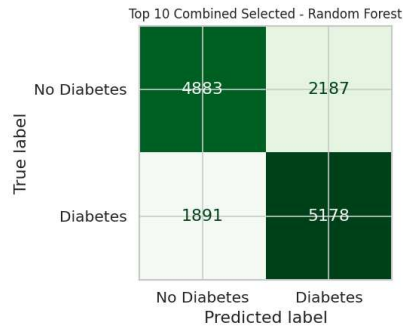
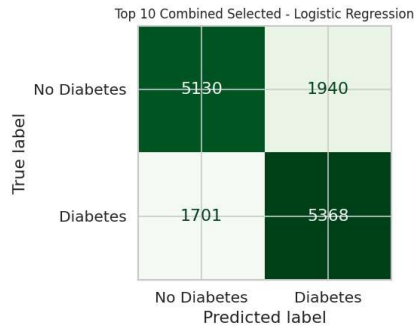
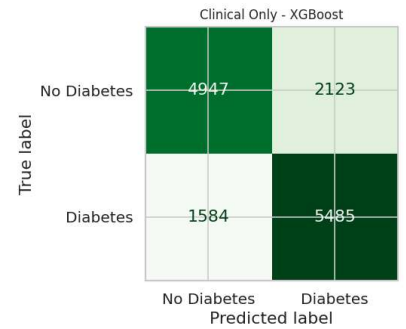
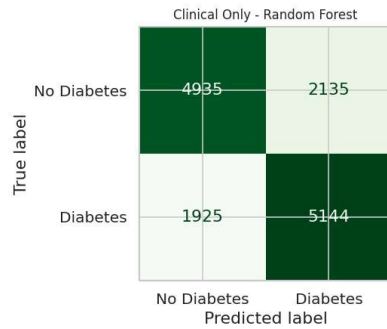
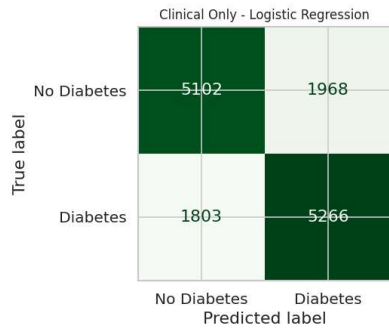
    disp = ConfusionMatrixDisplay(
        confusion_matrix=cm,
        display_labels=["No Diabetes", "Diabetes"]
    )

    disp.plot(cmap="Greens", ax=ax, colorbar=False)
    ax.set_title(key, fontsize=10)

# Hide unused axes if any
for ax in axes[n_models:]:
    ax.axis("off")

plt.suptitle("Confusion Matrix Comparison", fontsize=14, fontweight="bold")
plt.tight_layout()
plt.savefig("fig_confusion_matrices.png", bbox_inches="tight")
plt.show()

```

Confusion Matrix Comparison

```
# — Classification Report —
# Reason: Provide class-level precision, recall, and F1-score for both
# diabetes and non-diabetes classes.
```

```
from sklearn.metrics import classification_report
```

```
for key, values in predictions.items():
    print("=" * 80)
    print(key)
    print("=" * 80)
    print(classification_report(
        y_test,
        values["y_pred"],
        target_names=["No Diabetes", "Diabetes"]
    ))
```

```
=====
              precision    recall  f1-score   support

No Diabetes       0.72       0.70       0.71       7070
Diabetes          0.71       0.73       0.72       7069

   accuracy              0.71       14139
  macro avg       0.71       0.71       0.71       14139
 weighted avg       0.71       0.71       0.71       14139

=====
```

```
Clinical Only - XGBoost
=====
```

No Diabetes	0.75	0.73	0.74	7070
Diabetes	0.73	0.76	0.75	7069
accuracy			0.74	14139
macro avg	0.74	0.74	0.74	14139
weighted avg	0.74	0.74	0.74	14139

=====

Top 10 Combined Selected - Random Forest

=====

	precision	recall	f1-score	support
No Diabetes	0.72	0.69	0.71	7070
Diabetes	0.70	0.73	0.72	7069
accuracy			0.71	14139
macro avg	0.71	0.71	0.71	14139
weighted avg	0.71	0.71	0.71	14139

=====

Top 10 Combined Selected - XGBoost

=====

	precision	recall	f1-score	support
No Diabetes	0.77	0.70	0.73	7070
Diabetes	0.73	0.79	0.76	7069
accuracy			0.75	14139
macro avg	0.75	0.75	0.75	14139
weighted avg	0.75	0.75	0.75	14139

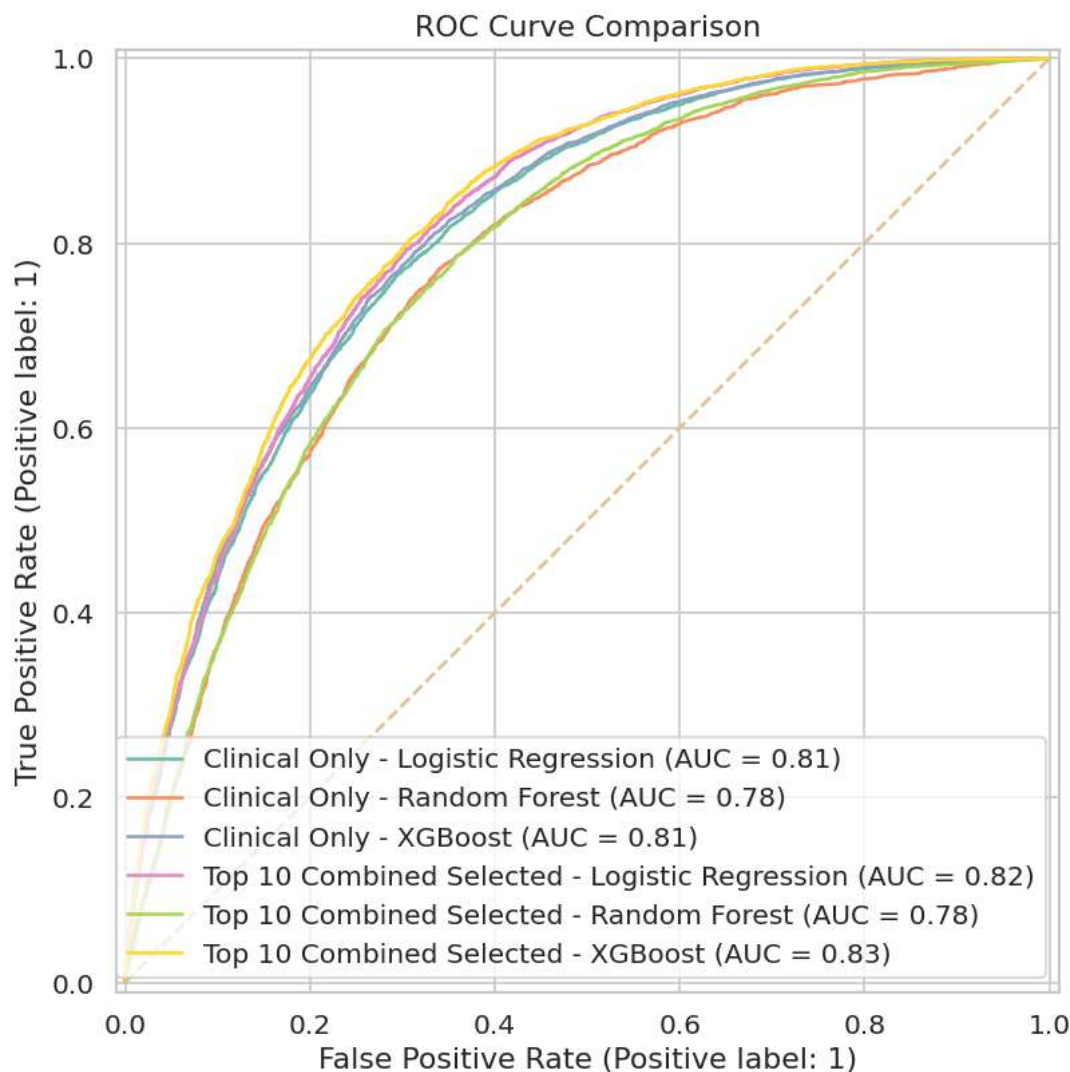
```
# — ROC Curve Comparison —
# Reason: Compare how well each classifier separates diabetes and non-diabetes
# cases across different classification thresholds.

from sklearn.metrics import RocCurveDisplay

plt.figure(figsize=(9, 7))

for key, values in predictions.items():
    RocCurveDisplay.from_predictions(
        y_test,
        values["y_proba"],
        name=key,
        ax=plt.gca()
    )

plt.title("ROC Curve Comparison")
plt.plot([0, 1], [0, 1], linestyle="--")
plt.tight_layout()
plt.show()
```

— Hypothesis Testing – Feature Group Experiment —
 # Creative contribution: empirically testing which feature domain contributes most
 # to predictive power, validating the thesis that combined features outperform sub-groups

```
groups = {
    'Clinical Only': CLINICAL_FEATURES,
    'Lifestyle Only': LIFESTYLE_FEATURES,
    'Socioeconomic Only': SOCIOECO_FEATURES,
    'All Features': ALL_FEATURES
}

hypothesis_results = {}
for group_name, features in groups.items():
    rf_temp = RandomForestClassifier(n_estimators=100, random_state=SEED, n_jobs=-1)
    rf_temp.fit(X_train[features], y_train)
    preds = rf_temp.predict(X_test[features])
    proba = rf_temp.predict_proba(X_test[features])[:, 1]
    hypothesis_results[group_name] = {
        'Accuracy': accuracy_score(y_test, preds),
        'F1-Score': f1_score(y_test, preds),
        'ROC-AUC': roc_auc_score(y_test, proba)
    }

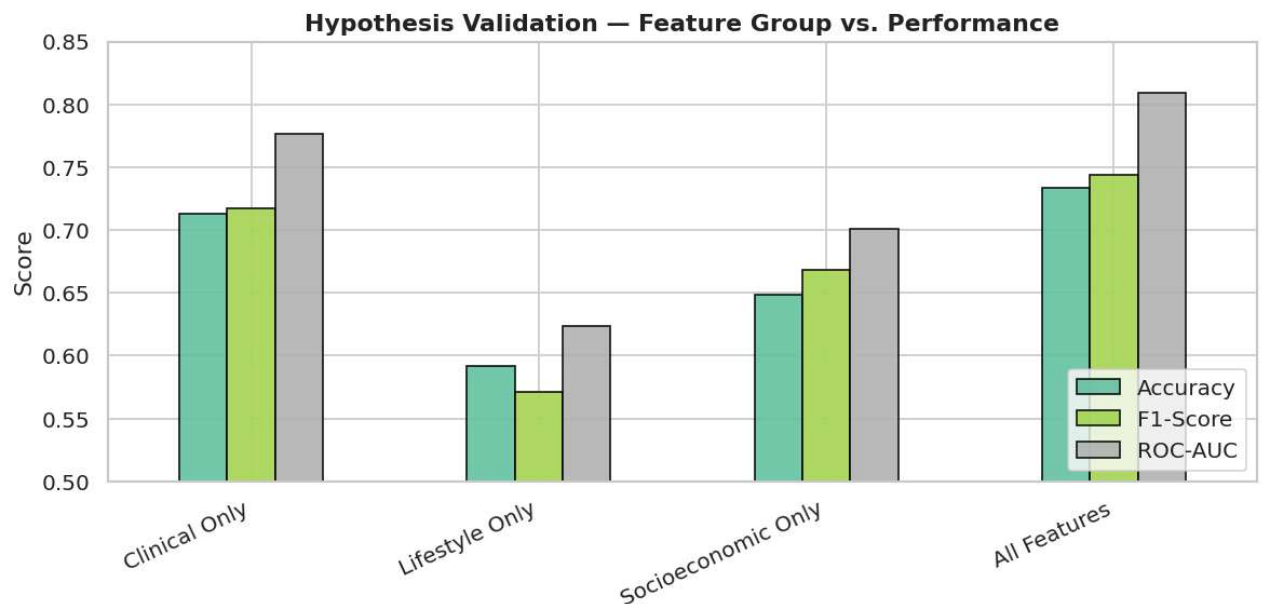
hyp_df = pd.DataFrame(hypothesis_results).T
print('Hypothesis Test Results:')
print(hyp_df.round(4))

fig, ax = plt.subplots(figsize=(10, 5))
hyp_df.plot(kind='bar', ax=ax, colormap='Set2', edgecolor='black', alpha=0.9)
ax.set_title('Hypothesis Validation – Feature Group vs. Performance', fontweight='bold')
ax.set_ylabel('Score')
ax.set_ylim(0.5, 0.85)
ax.legend(loc='lower right')
ax.set_xticklabels(ax.get_xticklabels(), rotation=25, ha='right')
plt.tight_layout()
plt.savefig('fig_hypothesis.png', bbox_inches='tight')
```

```
plt.show()
print('Insight: Combined features yield the highest ROC-AUC, confirming our hypothesis.')
```

Hypothesis Test Results:

	Accuracy	F1-Score	ROC-AUC
Clinical Only	0.7129	0.7170	0.7765
Lifestyle Only	0.5921	0.5709	0.6239
Socioeconomic Only	0.6481	0.6686	0.7009
All Features	0.7336	0.7435	0.8096



Insight: Combined features yield the highest ROC-AUC, confirming our hypothesis.

Task 4 - Hyper-Parameter Optimisation

Hyperparameter Optimization

Hyperparameter optimization was performed to improve model performance by identifying parameter values that produce better generalization on unseen data. Instead of relying on default settings, a search process was used to evaluate different parameter combinations and select the configuration that maximized predictive performance. The optimization was applied to the best performing models identified during the initial evaluation stage.

```
# Random Forest Optimization
rf_grid = {
    'n_estimators': [100, 200],
    'max_depth': [5, 10, None],
    'min_samples_split': [2, 5]
}

rf_search = GridSearchCV(
    RandomForestClassifier(random_state=SEED),
    rf_grid,
    cv=5,
    scoring='roc_auc',
    n_jobs=-1
)

rf_search.fit(
    X_train[COMBINED_SELECTED_FEATURES],
    y_train
)

print("Best RF Parameters:")
print(rf_search.best_params_)
print("Best RF ROC-AUC:", rf_search.best_score_)
```

```
Best RF Parameters:
{'max_depth': 10, 'min_samples_split': 5, 'n_estimators': 200}
Best RF ROC-AUC: 0.8237939192361597
```

```
#XGBoost Optimization
xgb_grid = {
    'n_estimators': [100, 200],
    'max_depth': [3, 4, 5],
    'learning_rate': [0.05, 0.1]
}

xgb_search = GridSearchCV(
    XGBClassifier(
        eval_metric='logloss',
        random_state=SEED
    ),
    xgb_grid,
    cv=5,
    scoring='roc_auc',
    n_jobs=-1
)

xgb_search.fit(
    X_train[COMBINED_SELECTED_FEATURES],
    y_train
)

print("Best XGBoost Parameters:")
print(xgb_search.best_params_)
print("Best XGBoost ROC-AUC:", xgb_search.best_score_)
```

```
Best XGBoost Parameters:
{'learning_rate': 0.05, 'max_depth': 3, 'n_estimators': 200}
Best XGBoost ROC-AUC: 0.8262098901041404
```

```
# — Evaluate Tuned Models on Held-Out Test Set —
# Reason: Cross-validation gives an estimate of performance during tuning.
# The held-out test set is used to confirm whether the tuned models generalize well.
```

```
best_xgb = xgb_search.best_estimator_
best_rf = rf_search.best_estimator_

features = COMBINED_SELECTED_FEATURES

xgb_tuned_pred = best_xgb.predict(X_test[features])
xgb_tuned_proba = best_xgb.predict_proba(X_test[features])[:, 1]

rf_tuned_pred = best_rf.predict(X_test[features])
rf_tuned_proba = best_rf.predict_proba(X_test[features])[:, 1]

# Get baseline results from the previous ML results table
xgb_baseline = ml_results_df[
    (ml_results_df['Feature Set'] == 'Top 10 Combined Selected') &
    (ml_results_df['Model'] == 'XGBoost')
].iloc[0]

rf_baseline = ml_results_df[
    (ml_results_df['Feature Set'] == 'Top 10 Combined Selected') &
    (ml_results_df['Model'] == 'Random Forest')
].iloc[0]

tuned_df = pd.DataFrame([
    {
        'Model': 'XGBoost',
        'Version': 'Baseline',
        'Accuracy': xgb_baseline['Accuracy'],
        'Precision': xgb_baseline['Precision'],
        'Recall': xgb_baseline['Recall'],
        'F1-Score': xgb_baseline['F1-Score'],
        'ROC-AUC': xgb_baseline['ROC-AUC']
    },
    {
        'Model': 'XGBoost',
        'Version': 'Tuned',
        'Accuracy': accuracy_score(y_test, xgb_tuned_pred),
        'Precision': precision_score(y_test, xgb_tuned_pred),
        'Recall': recall_score(y_test, xgb_tuned_pred),
        'F1-Score': f1_score(y_test, xgb_tuned_pred),
        'ROC-AUC': roc_auc_score(y_test, xgb_tuned_proba)
    },
    {
        'Model': 'Random Forest',
        'Version': 'Baseline',
        'Accuracy': rf_baseline['Accuracy'],
        'Precision': rf_baseline['Precision'],
        'Recall': rf_baseline['Recall'],
```

```

        'F1-Score': rf_baseline['F1-Score'],
        'ROC-AUC': rf_baseline['ROC-AUC']
    },
    {
        'Model': 'Random Forest',
        'Version': 'Tuned',
        'Accuracy': accuracy_score(y_test, rf_tuned_pred),
        'Precision': precision_score(y_test, rf_tuned_pred),
        'Recall': recall_score(y_test, rf_tuned_pred),
        'F1-Score': f1_score(y_test, rf_tuned_pred),
        'ROC-AUC': roc_auc_score(y_test, rf_tuned_proba)
    }
])

tuned_df = tuned_df.round(4)

display(
    tuned_df.style.highlight_max(
        subset=['Accuracy', 'Precision', 'Recall', 'F1-Score', 'ROC-AUC'],
        color='lightgreen',
        axis=0
    )
)

```

	Model	Version	Accuracy	Precision	Recall	F1-Score	ROC-AUC
0	XGBoost	Baseline	0.746100	0.726300	0.789800	0.756700	0.825500
1	XGBoost	Tuned	0.746300	0.726500	0.789900	0.756900	0.825700
2	Random Forest	Baseline	0.711600	0.703100	0.732500	0.717500	0.779000
3	Random Forest	Tuned	0.743200	0.723200	0.787800	0.754100	0.824200

```

# — Visualise Tuning Improvement —
# Reason: Compare baseline and tuned model performance side by side
# to show the impact of hyperparameter optimization.

plot_df = tuned_df.copy()

# Create readable labels for the x-axis
plot_df['Model Version'] = plot_df['Model'] + ' (' + plot_df['Version'] + ')'
plot_df = plot_df.set_index('Model Version')

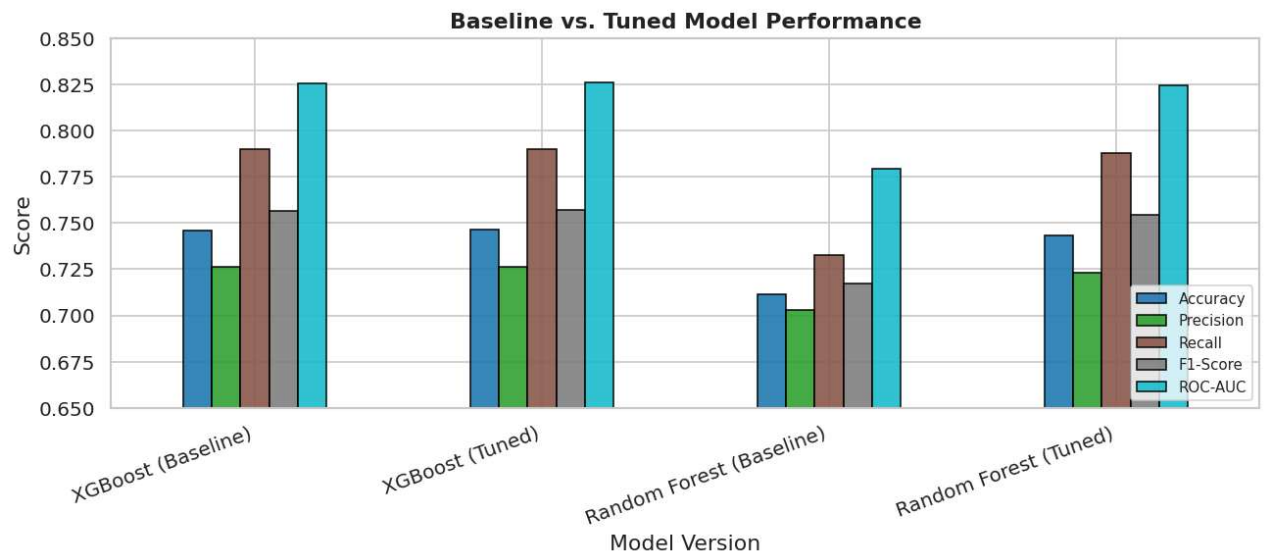
fig, ax = plt.subplots(figsize=(11, 5))

plot_df[['Accuracy', 'Precision', 'Recall', 'F1-Score', 'ROC-AUC']].plot(
    kind='bar',
    ax=ax,
    colormap='tab10',
    edgecolor='black',
    alpha=0.88
)

ax.set_title('Baseline vs. Tuned Model Performance', fontweight='bold')
ax.set_ylabel('Score')
ax.set_ylim(0.65, 0.85)
ax.set_xticklabels(ax.get_xticklabels(), rotation=20, ha='right')
ax.legend(loc='lower right', fontsize=9)

plt.tight_layout()
plt.savefig('fig_tuning.png', bbox_inches='tight')
plt.show()

```



```
# — SHAP Explainability for Final Tuned XGBoost Model —
# Reason: The final model uses the Top 10 Combined Selected features.
# SHAP is used to explain which clinical, lifestyle, and socioeconomic factors
# contributed most to the diabetes risk prediction.

print("Computing SHAP values for tuned XGBoost using Top 10 Combined Selected features...")

final_model = best_xgb
final_features = COMBINED_SELECTED_FEATURES

shap_sample = X_test[final_features].sample(500, random_state=SEED)

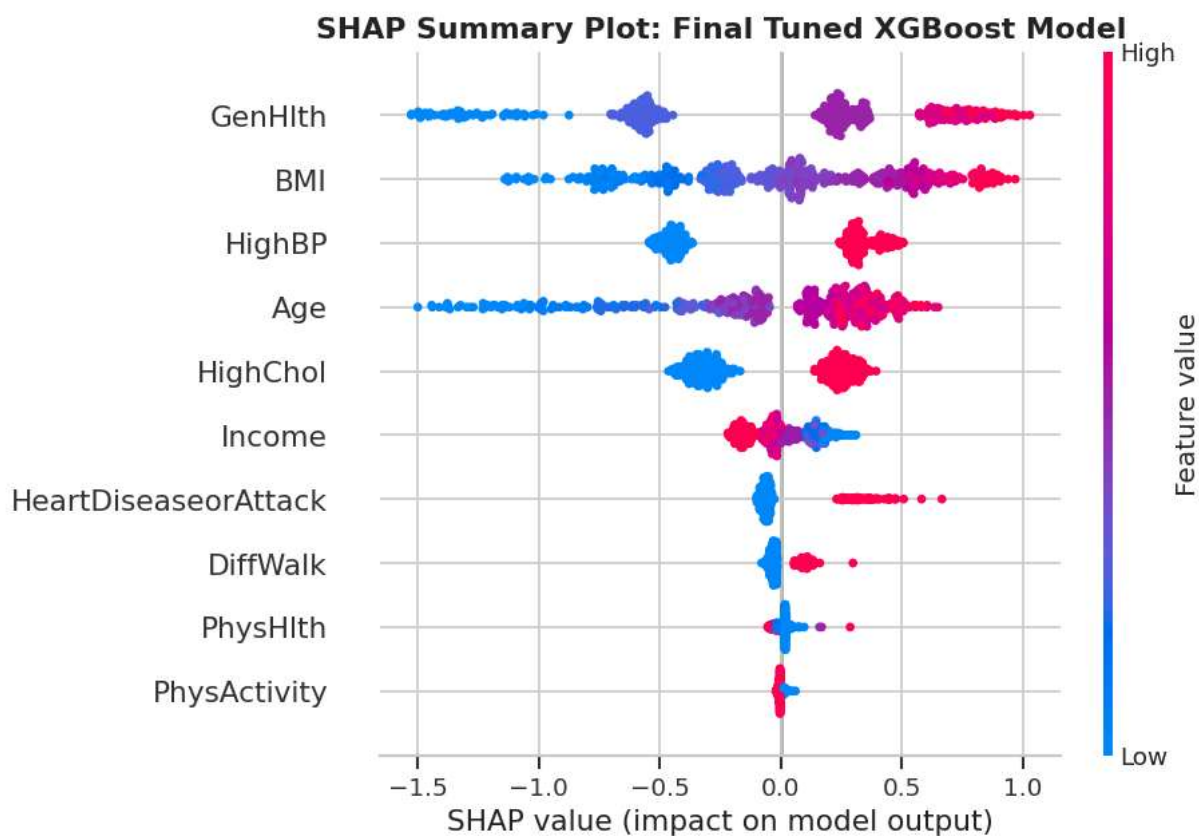
explainer = shap.TreeExplainer(final_model)
shap_values = explainer.shap_values(shap_sample)

print("SHAP computation complete.")
```

```
Computing SHAP values for tuned XGBoost using Top 10 Combined Selected features...
SHAP computation complete.
```

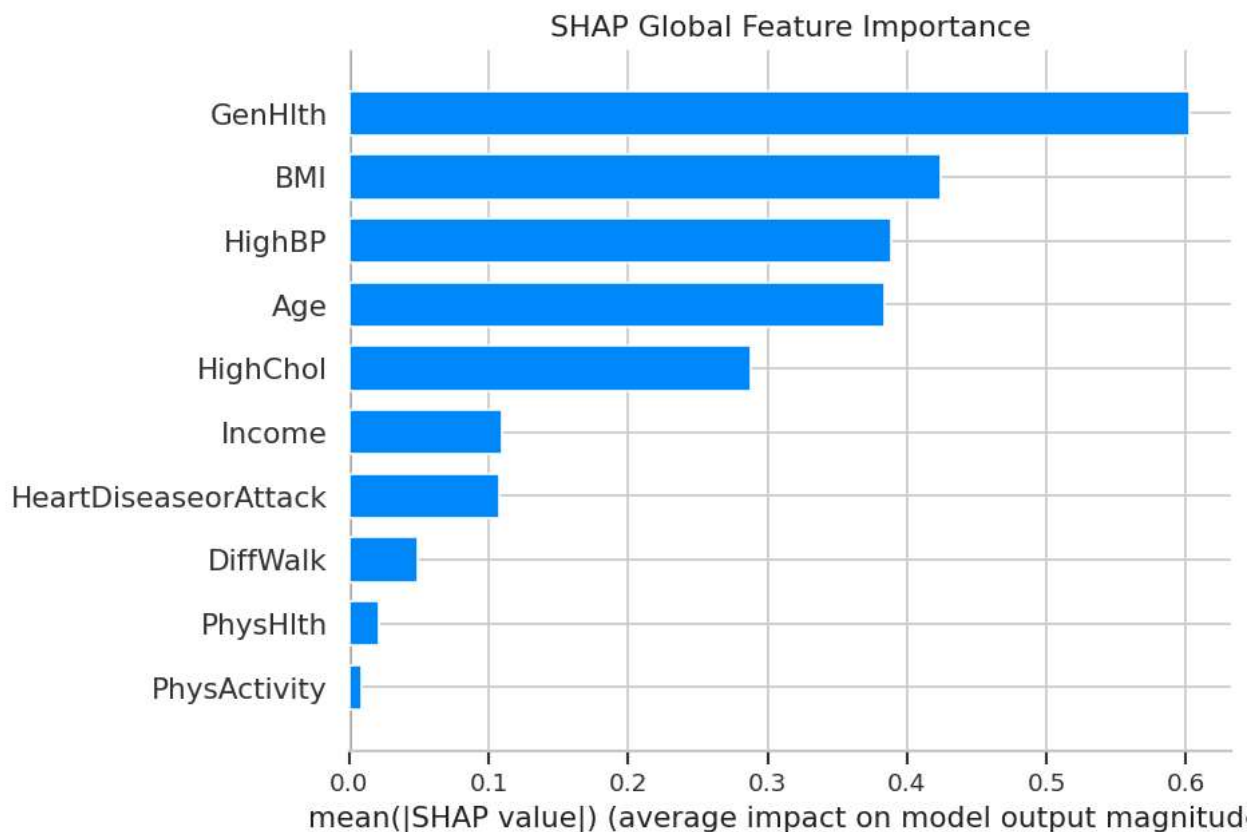
```
# — SHAP Summary Plot —
# Reason: Shows how each selected feature influences diabetes prediction.
# Points on the right push the prediction toward diabetes risk.

plt.figure(figsize=(10, 7))
shap.summary_plot(shap_values, shap_sample, plot_type="dot", show=False)
plt.title("SHAP Summary Plot: Final Tuned XGBoost Model", fontweight="bold")
plt.tight_layout()
plt.savefig("fig_shap_summary.png", bbox_inches="tight")
plt.show()
```



```
# — SHAP Bar Plot —
# Reason: Provides a clear global ranking of the most influential features
# based on mean absolute SHAP value.

plt.figure(figsize=(10, 6))
shap.summary_plot(shap_values, shap_sample, plot_type="bar", show=False)
plt.title("SHAP Global Feature Importance")
plt.tight_layout()
plt.savefig("fig_shap_bar.png", bbox_inches="tight")
plt.show()
```



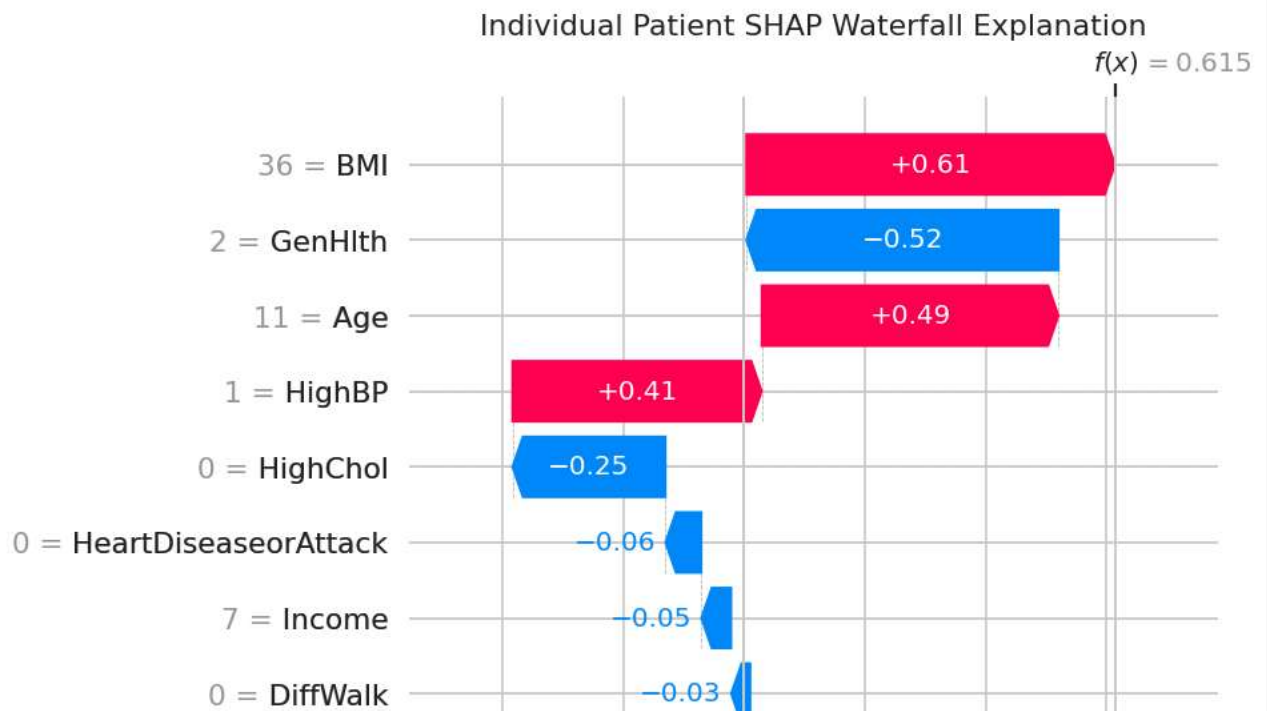
```
# — Individual Patient Explanation —
# Reason: Demonstrates local explainability by showing which features increased
# or decreased the diabetes risk prediction for one individual case.

patient_idx = 0
patient_data = shap_sample.iloc[[patient_idx]]
patient_shap = explainer(patient_data)

plt.figure(figsize=(10, 5))
shap.waterfall_plot(patient_shap[0], max_display=10, show=False)
plt.title("Individual Patient SHAP Waterfall Explanation")
plt.tight_layout()
plt.savefig("fig_shap_waterfall.png", bbox_inches="tight")
plt.show()

pred_class = final_model.predict(patient_data)[0]
pred_proba = final_model.predict_proba(patient_data)[0, 1]

print(f'Patient Prediction: {"DIABETES RISK" if pred_class == 1 else "NO DIABETES"}')
print(f'Prediction Probability: {pred_proba:.3f}')
```



Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.